

M Project 实现路径完全指南

工业定长送料与定位执行单元—代码理解手册

文档版本: 1.0

日期: 2026 年 7 月 10 日

目录

1. 项目概述
2. 学习路径规划
3. 第一阶段：入口与启动
4. 第二阶段：主循环机制
5. 第三阶段：状态机核心
6. 第四阶段：初始化流程
7. 第五阶段：CAN 通信处理
8. 第六阶段：上位机协议
9. 第七阶段：运动控制
10. 第八阶段：安全监测
11. 第九阶段：中断系统
12. 第十阶段：完整数据流
13. 附录：常见问题

1. 项目概述

1.1 项目背景

本项目是一个工业定长送料与定位执行单元，核心功能是通过 STM32G474 主控板控制 ZDT X57S 闭环步进电机，实现精准的位置控制和自动化送料。

1.2 系统架构



核心思想：主控作为双向消息中介，负责指令转发、状态收集、逻辑编排、异常处理。

1.3 分层架构

层级	模块	职责
应用层	state machine, protocol, motion, safety	业务逻辑
驱动层	X V2	电机驱动库
HAL 层	fdcan, usart, tim, gpio	硬件抽象
硬件层	STM32G474 + ZDT X57S	物理设备

1.4 模块职责表

模块	文件	职责	关键函数
状态机	state machine.c/h	系统状态管理、事件处理	SM Run(), SM PostEvent()
协议	protocol.c/h	上位机通信协议	Protocol Parse(), Protocol SendEvent()
运动控制	motion.c/h	工艺动作封装	Motion DoHoming(), Motion MoveTo()
安全监测 参数管理	safety.c/h motor params.c/h	电机状态检查 参数读写封装	Safety Check() MotorParam ReadRequest()

2. 学习路径规划

2.1 学习原则

循序渐进：从入口开始，理解主循环，深入状态机，逐个模块学习，最后串联数据流。

2.2 学习阶段总览

阶段	内容	时间	目标
1	入口与启动	5 分钟	理解系统从哪里开始
2	主循环机制	10 分钟	理解系统如何运行
3	状态机核心	15 分钟	理解状态转换逻辑
4	初始化流程	10 分钟	理解上电自动流程
5	CAN 通信处理	15 分钟	理解电机数据处理
6	上位机协议	10 分钟	理解命令解析分发
7	运动控制	10 分钟	理解动作执行
8	安全监测	5 分钟	理解异常检测
9	中断系统	10 分钟	理解中断分工
10	完整数据流	10 分钟	串联所有模块

总学习时间：约 100 分钟

2.3 学习检查点

阶段	检查问题
1	系统从哪个函数开始？初始化了哪些外设？
2	主循环每一轮做了什么？
3	状态机有哪些状态？如何转换？
4	上电后系统自动做了什么？
5	CAN 数据如何接收和处理？
6	上位机命令如何解析？
7	具体动作如何执行？
8	异常如何检测？
9	中断和主循环如何分工？
10	完整数据流是怎样的？

3. 第一阶段：入口与启动

3.1 学习目标

理解系统从哪里开始，做了什么初始化工作。

3.2 关键文件

Core/Src/main.c -> main() 函数（第28-95行）

3.3 main() 函数结构

```
int main(void)
{
    // 1. HAL 初始化
    HAL_Init();
    SystemClock Config();

    // 2. 外设初始化
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_FDCAN1_Init();
    MX_USART1_UART_Init();
    MX_TIM6_Init();

    // 3. 用户代码初始化
    Protocol_Init();           // 协议解析器初始化
    HAL_FDCAN_Start(&hfdcan1); // 启动 FDCAN
    HAL_TIM_Base_Start_IT(&htim6); // 启动 TIM6

    SM_Init();                 // 状态机初始化

    // 4. 主循环
    while (1) {
        SM_Run();
    }
}
```

3.4 初始化顺序

步骤	函数	作用
1	HAL_Init()	初始化 HAL 库
2	SystemClock Config()	配置系统时钟 (170MHz)
3	MX_GPIO_Init()	初始化 GPIO 引脚
4	MX_DMA_Init()	初始化 DMA 控制器
5	MX_FDCAN1_Init()	初始化 FDCAN(500Kbps)
6	MX_USART1_UART_Init()	初始化 USART1(115200)
7	MX_TIM6_Init()	初始化 TIM6(1kHz)
8	Protocol_Init()	协议解析器初始化
9	SM_Init()	状态机初始化

关键理解：系统启动后，所有逻辑都在主循环的 SM_Run() 中执行。

3.5 代码位置速查

内容	文件	行号
main() 入口	main.c	28
外设初始化	main.c	31-36
协议初始化	main.c	49
状态机初始化	main.c	83
主循环	main.c	92-94

4. 第二阶段：主循环机制

4.1 学习目标

理解系统每一轮循环做了什么，如何处理各种输入。

4.2 关键文件

App/Src/state machine.c -> SM Run() 函数（第1093-1207行）

4.3 主循环流程

SM Run() 主循环每一轮做 5 件事：

1. 消费 **USART** 接收缓冲区
 - 从环形缓冲区取出字节
 - 调用 Protocol Parse() 逐字节解析协议帧
2. 消费 **CAN** 接收缓冲区
 - 从环形缓冲区取出 CAN 帧
 - 调用 SM ProcessCanFrame() 处理电机返回数据
3. 处理急停按钮
 - 检测按钮状态
 - 触发 EV ESTOP TRIGGERED 事件
4. 待应答命令超时检查
 - 检查每条待应答命令是否超时
 - 超时则触发错误处理
5. 周期性安全检查（每 **100ms**）
 - 调用 Safety Check() 读取电机状态标志位
 - 检测异常并触发错误事件

4.4 关键代码解析

消费 **USART** 缓冲区（第 1100-1104 行）

```
while (g usart rx tail != g usart rx head) {
    uint8 t byte = g usart rx buf[g usart rx tail];
    g usart rx tail = (g usart rx tail + 1) % USART_RX_BUF_SIZE;
    Protocol Parse(byte);
}
```

关键理解：USART 使用环形缓冲区，中断只负责收，主循环负责解析。

消费 **CAN** 缓冲区（第 1108-1110 行）

```
CanRxFrame t rxFrame;
while (CanRxBuf Pop(&rxFrame)) {
    SM ProcessCanFrame(&rxFrame);
}
```

关键理解：CAN 使用环形缓冲区，中断只负责收，主循环负责处理。

周期性安全检查（第 1160-1168 行）

```
static uint32 t last safety check = 0;
if (g sys tick - last safety check >= 100) {
    last safety check = g sys tick;
    ErrorCode t err = Safety Check();
    if (err != ERR NONE) {
        s last error = err;
        SM PostEvent(EV ERROR DETECTED);
    }
}
```

关键理解：安全检查每 100ms 执行一次，通过读取电机状态标志位检测异常。

4.5 主循环设计哲学

主循环是一个“事件消费循环”，不断从各种缓冲区取出事件并处理：- 中断与主循环的解耦 - 事件的顺序处理 - 避免中断嵌套

4.6 代码位置速查

内容	文件	行号
SM Run() 函数入口	state machine.c	1093
消费 USART 缓冲区	state machine.c	1100-1104
消费 CAN 缓冲区	state machine.c	1108-1110
急停按钮处理	state machine.c	1113-1120
超时检查	state machine.c	1123-1141
安全检查	state machine.c	1160-1168

5. 第三阶段：状态机核心

5.1 学习目标

理解状态机的状态定义、事件定义和状态转换逻辑。

5.2 关键文件

App/Inc/state machine.h -> 状态枚举（第10-18行）
App/Inc/state machine.h -> 事件枚举（第23-39行）
App/Src/state machine.c -> SM PostEvent() 函数（第916-1091行）

5.3 七种系统状态

状态	值	说明	允许的命令
INIT	0	上电初始化，自动回零	无
IDLE	1	待机就绪	HOME, MOVE, CYCLE
HOMING	2	回零执行中	STOP
RUNNING	3	单次动作执行中	STOP
CYCLING	4	循环运行中	STOP, CYCLE STOP
ERROR	5	异常暂停	RESET ERROR
ESTOP	6	急停锁定	RESET ERROR(释放后)

5.4 十四种系统事件

事件	值	来源	触发条件
EV NONE	0	-	-
EV SYSTEM READY	1	初始化	初始化完成
EV CMD HOME	2	上位机	收到 HOME 命令
EV CMD MOVE	3	上位机	收到 MOVE 命令
EV CMD CYCLE	4	上位机	收到 CYCLE 命令
EV CMD STOP	5	上位机	收到 STOP 命令
EV CMD RESET ERROR	6	上位机	收到 RESET 命令
EV HOME DONE	7	CAN 应答	收到 0x9F(回零完成)
EV HOME FAILED	8	CAN 应答/超时	收到 0xE2 或超时
EV MOVE DONE	9	CAN 应答	收到 0x9F(动作完成)
EV CYCLE NEXT	10	状态机	循环下一点
EV CYCLE DONE	11	状态机	循环完成
EV ERROR DETECTED	12	安全检查	检测到电机异常
EV ESTOP TRIGGERED	13	硬件中断	急停按钮按下
EV ESTOP RELEASED	14	硬件中断	急停按钮释放

5.5 状态转换表

当前状态	事件	目标状态	动作
INIT	EV SYSTEM READY	IDLE	无
IDLE	EV CMD HOME	HOMING	触发回零
IDLE	EV CMD MOVE	RUNNING	发送移动命令
IDLE	EV CMD CYCLE	CYCLING	启动循环
HOMING	EV HOME DONE	IDLE	完成回零
HOMING	EV HOME FAILED	ERROR	记录错误
HOMING	EV CMD STOP	IDLE	中断回零
RUNNING	EV MOVE DONE	IDLE	完成移动
RUNNING	EV CMD STOP	IDLE	停止移动
CYCLING	EV MOVE DONE	CYCLING	等待停留
CYCLING	EV CYCLE NEXT	CYCLING	下一点
CYCLING	EV CYCLE DONE	IDLE	循环完成
ERROR	EV CMD RESET ERROR	IDLE	清除错误
ESTOP	EV CMD RESET ERROR	IDLE	释放后清除

5.6 状态转换优先级

1. 最高优先级: EV ESTOP TRIGGERED - 可打断任何状态
2. 次高优先级: EV ERROR DETECTED - 可打断运动状态
3. 普通优先级: 其他事件按当前状态处理

5.7 SM PostEvent() 函数逻辑

```
void SM PostEvent(SystemEvent t event)
{
    if (event == EV NONE) return;

    // 最高优先级: 急停可打断任何状态
    if (event == EV ESTOP TRIGGERED) {
        if (g sm state != SYSTEM ESTOP) {
            SM EnterState(SYSTEM ESTOP);
        }
    }
}
```

```

        return;
    }

    // 次高优先级：错误可打断运动状态
    if (event == EV_ERROR DETECTED) {
        if (g sm state != SYSTEM ESTOP && g sm state != SYSTEM ERROR) {
            SM EnterState(SYSTEM ERROR);
        }
        return;
    }

    // 按当前状态处理事件
    switch (g sm state) {
        case SYSTEM IDLE:
            if (event == EV_CMD HOME) SM EnterState(SYSTEM HOMING);
            else if (event == EV_CMD MOVE) SM EnterState(SYSTEM RUNNING);
            else if (event == EV_CMD CYCLE) SM EnterState(SYSTEM CYCLING);
            break;
        // ... 其他状态处理
    }
}

```

5.8 代码位置速查

内容	文件	行号
状态枚举定义	state machine.h	10-18
事件枚举定义	state machine.h	23-39
SM PostEvent()	state machine.c	916-1091
SM EnterState()	state machine.c	856-914

6. 第四阶段：初始化流程

6.1 学习目标

理解系统上电后自动做了什么，特别是回零流程。

6.2 关键文件

App/Src/state machine.c -> SM Init() (第834-854行)
 App/Src/state machine.c -> SM EnterState() (第856-914行)

6.3 SM Init() 函数

```

void SM Init(void)
{
    CanRxBuf Init();           // 初始化 CAN 接收缓冲区
    Safety Init();             // 初始化安全模块

    // 清空待应答表
    for (uint8 t i = 0; i < PENDING MAX; i++) {
        s pending[i].active = false;
    }
}

```

```

s last error = ERR NONE;
g estop triggered = false;
g sys tick = 0;

// 注册协议帧处理器
Protocol RegisterHandler(ProtocolFrameHandler);

// 进入 INIT 状态
SM EnterState(SYSTEM INIT);
}

```

6.4 INIT 状态处理

```

case SYSTEM INIT:
    // 1. 使能电机
    Motion Enable();
    HAL Delay(100);

    // 2. 配置电机 Response=Both
    SM InitMotorConfig();

    // 3. 自动进入 HOMING 状态
    SM EnterState(SYSTEM HOMING);
    return;

```

6.5 HOMING 状态处理

```

case SYSTEM HOMING:
    // 1. 重置回零监控器
    SM ResetHomingMonitor();

    // 2. 通知上位机：回零已启动
    Protocol SendEvent(PROTO EVT HOMING START, 0x9A, 0x00, NULL, 0);

    // 3. 触发碰撞回零
    Motion DoHoming();

    // 4. 注册超时跟踪
    SM TrackPendingCmd(0x9A); // 跟踪 ACK
    SM TrackActionCmd(0x9A, 180000); // 跟踪完成，最长 180 秒
    break;

```

6.6 回零流程

```

上电
|
v
INIT状态
|-> 使能电机
|-> 配置电机 Response=Both
v
HOMING状态
|-> 触发碰撞回零
|-> 注册超时跟踪(180秒)
|
|-> 收到0x9F -----> HOME DONE -> IDLE
|-> 电流>=450mA ----> 完成回零 -> IDLE

```

```
|-> 收到0xE2 -----> HOME FAILED -> ERROR
|-> 超时180秒 -----> HOME FAILED -> ERROR
```

链路 A: 上电自动初始化与回零, 无需上位机干预。

6.7 Motion DoHoming() 函数

```
void Motion DoHoming(void)
{
    if (s homing mode == 2U) {
        // 方式 2: 速度模式 + 电流限制 (碰撞回零)
        X V2 Vel LC Control(MOTOR ADDR, s homing dir, 100,
                           (float)s homing vel rpm, false,
                           s homing current limit ma);
    } else {
        // 方式 1/3: 电机内置回零
        X V2 Origin Modify SL RP(MOTOR ADDR, false, HOMING COLLISION RELEASE 01DEG);
        HAL Delay(150);
        X V2 Origin Trigger Return(MOTOR ADDR, s homing mode, false);
    }
}
```

6.8 代码位置速查

内容	文件	行号
SM Init()	state machine.c	834-854
INIT 状态处理	state machine.c	863-871
HOMING 状态处理	state machine.c	876-888
Motion DoHoming()	motion.c	45-56

7. 第五阶段: CAN 通信处理

7.1 学习目标

理解电机返回的数据是如何接收和处理的。

7.2 关键文件

App/Src/state machine.c -> SM ProcessCanFrame() (第483-635行)

Core/Src/stm32g4xx_it.c -> HAL FDCAN RxFifo0Callback() (第258-275行)

7.3 CAN 接收流程

中断接收

```
void HAL FDCAN RxFifo0Callback(FDCAN HandleTypeDef hfdcan, uint32_t RxFifo0ITs)
{
    FDCAN RxHeaderTypeDef rxHeader;
    uint8_t rxData[8];

    if ((RxFifo0ITs & FDCAN_IT_RX_FIFO0_NEW_MESSAGE) != 0U) {
        if (HAL FDCAN GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &rxHeader, rxData) == HAL_OK) {
            g can rx count++;
            uint8_t dlc bytes = (uint8_t)(rxHeader.DataLength);
        }
    }
}
```

```

        CanRxBuf Push(rxHeader.Identifier, dlc bytes, rxData);
    }
}

```

环形缓冲区

```

// 写入 (中断上下文)
bool CanRxBuf Push(uint32 t id, uint8 t dlc, uint8 t data)
{
    uint8 t next head = (g can rx buf.head + 1) % CAN RX BUF SIZE;
    if (next head == g can rx buf.tail) {
        g can rx drop count++;
        return false;
    }
    // 存储数据...
    g can rx buf.head = next head;
    return true;
}

// 读取 (主循环)
bool CanRxBuf Pop(CanRxFrame t frame)
{
    if (g can rx buf.head == g can rx buf.tail) return false;
    // 读取数据...
    g can rx buf.tail = (g can rx buf.tail + 1) % CAN RX BUF SIZE;
    return true;
}

```

7.4 SM ProcessCanFrame() 函数

```

static void SM ProcessCanFrame(CanRxFrame t frame)
{
    uint8 t func = frame->data[0];    // 功能码
    uint8 t byte1 = frame->data[1];   // 应答码或数据

    // 检查校验字节 (最后一字节应为 0x6B)
    if (frame->data[frame->dlc - 1] != 0x6B) return;

    // 区分数据帧和应答帧
    if (IsParamDataFunc(func)) {
        SM ProcessMotorData(func, &frame->data[1], data len);
        return;
    }

    if (IsReplyCode(byte1)) {
        switch (byte1) {
            case 0x02: // 命令确认收到
                Protocol SendEvent(PROTO EVT ACK RECEIVED, func, 0x02, NULL, 0);
                SM ClearPendingCmd(func);
                break;

            case 0x9F: // 动作执行完成
                SM ClearPendingCmd(func);
                SM ClearActionCmd(func);
                if (func == 0x9A) {
                    SM PostEvent(EV HOME DONE);
                }
            }
        }
    }
}

```

```

    } else {
        SM PostEvent(EV MOVE DONE);
    }
    break;

case 0xE2: // 参数错误或保护触发
    Protocol SendEvent(PROTO EVT MOTOR ERROR, func, 0xE2, NULL, 0);
    SM PostEvent(EV ERROR DETECTED);
    break;
}
}
}
}

```

7.5 应答码说明

应答码	含义	处理方式
0x02	命令确认收到	清除待应答，发送 ACK 事件
0x9F	动作执行完成	清除待应答，触发完成事件
0xE2	参数错误/保护触发	发送错误事件，进入 ERROR
0xEE	格式/校验错误	发送错误事件
0x12	状态拒绝	发送错误事件

7.6 数据帧类型

功能码	参数名	说明	来源
0x20	总线电压	每 250ms 读取	Safety Check
0x21	相电流	每 10ms 读取	PollRealtimeTelemetry
0x35	实时转速	每 10ms 读取	PollRealtimeTelemetry
0x36	实时位置	每 10ms 读取	PollRealtimeTelemetry
0x37	位置误差	每 10ms 读取	PollRealtimeTelemetry
0x3A	电机状态标志	每 100ms 读取	Safety Check
0x3B	温度	每 250ms 读取	PollRealtimeTelemetry
0x3C	回零 + 电机状态	每 100ms 读取	Safety Check

7.7 代码位置速查

内容	文件	行号
CAN 中断回调	stm32g4xx_it.c	258-275
CanRxBuf Push()	state_machine.c	322-337
CanRxBuf Pop()	state_machine.c	339-350
SM ProcessCanFrame()	state_machine.c	483-635
SM ProcessMotorData()	state_machine.c	447-481

8. 第六阶段：上位机协议

8.1 学习目标

理解上位机命令是如何解析和分发的。

8.2 关键文件

App/Src/protocol.c -> Protocol Parse() (第95-164行)

App/Src/state machine.c -> ProtocolFrameHandler() (第653-828行)

8.3 协议帧格式

STX	LEN	PAYLOAD	CRC8	ETX
0xAA	1B	LEN bytes	1B	0x55

三种帧类型: - 命令帧 (上位机到 MCU): CMD + DATA - 响应帧 (MCU 到上位机): CMD + STATUS + DATA - 事件帧 (MCU 到上位机): 0x00 + EVT TYPE + FUNC + STATUS + DATA

8.4 Protocol Parse() 函数

```
void Protocol Parse(uint8 t byte)
{
    switch (g parse state) {
        case PARSE WAIT STX:
            if (byte == PROTO STX) { // 等待 0xAA
                g parse state = PARSE WAIT LEN;
            }
            break;

        case PARSE WAIT LEN:
            g rx len = byte; // 接收长度
            g parse state = PARSE WAIT CMD;
            break;

        case PARSE WAIT CMD:
            g rx buf[0] = byte; // 接收命令
            g rx count = 1;
            g parse state = PARSE WAIT DATA;
            break;

        case PARSE WAIT DATA:
            g rx buf[g rx count] = byte; // 接收数据
            g rx count++;
            if (g rx count >= g rx len) {
                g parse state = PARSE WAIT CRC;
            }
            break;

        case PARSE WAIT CRC:
            // 验证 CRC8
            if (CalCRC8(g rx buf, g rx len) == byte) {
                g parse state = PARSE WAIT ETX;
            } else {
                g parse state = PARSE WAIT STX;
            }
            break;

        case PARSE WAIT ETX:
            if (byte == PROTO ETX && g frame handler) {
                g frame handler(g rx buf[0], g rx buf + 1, g rx len - 1);
            }
    }
}
```

```

        g parse state = PARSE WAIT STX;
        break;
    }
}

```

8.5 ProtocolFrameHandler() 函数

```

static void ProtocolFrameHandler(uint8 t cmd, const uint8 t data, uint8 t len)
{
    // 运动命令在非 IDLE 状态拒绝
    if ((cmd == 0x01 || cmd == 0x02 || cmd == 0x03 || cmd == 0x05) &&
        g sm state != SYSTEM IDLE) {
        Protocol SendResponse(cmd, PROTO STAT BUSY, NULL, 0);
        return;
    }

    switch (cmd) {
    case 0x01: // CMD HOME
        Protocol SendResponse(cmd, PROTO STAT OK, NULL, 0);
        SM PostEvent(EV CMD HOME);
        break;

    case 0x02: // CMD MOVE ABS
    case 0x03: // CMD MOVE REL
        // 解析参数
        int32 t pos raw = ...;
        uint16 t vel raw = ...;
        uint16 t acc raw = ...;

        // 参数范围检查
        if (参数无效) {
            Protocol SendResponse(cmd, PROTO STAT FAIL, NULL, 0);
            break;
        }

        s pending pos deg = (float)pos raw / 10.0f;
        s pending vel rpm = (float)vel raw / 10.0f;
        s pending acc = acc raw;

        Protocol SendResponse(cmd, PROTO STAT OK, NULL, 0);
        SM PostEvent(EV CMD MOVE);
        break;

    case 0x04: // CMD STOP
        Protocol SendResponse(cmd, PROTO STAT OK, NULL, 0);
        SM PostEvent(EV CMD STOP);
        break;
    }
}

```

8.6 事件类型

EVT TYPE	名称	说明
0x80	ACK RECV	电机确认收到命令
0x81	ACT DONE	电机动作完成
0x82	MOTOR ERROR	电机返回错误

EVT TYPE	名称	说明
0x83	MOTOR DATA	电机数据透传
0x84	HOME START	回零已启动
0x85	HOME DONE	回零成功
0x86	HOME FAILED	回零失败
0x87	ESTOP	急停触发
0x88	TIMEOUT	通信超时

8.7 代码位置速查

内容	文件	行号
Protocol Parse()	protocol.c	95-164
Protocol SendResponse()	protocol.c	170-198
Protocol SendEvent()	protocol.c	229-259
ProtocolFrameHandler()	state machine.c	653-828

9. 第七阶段：运动控制

9.1 学习目标

理解具体运动动作是如何执行的。

9.2 关键文件

App/Src/motion.c -> Motion DoHoming() (第45-56行)
App/Src/motion.c -> Motion MoveTo() (第103-119行)
App/Src/motion.c -> Motion CycleNextPoint() (第186-216行)

9.3 回零函数

```
void Motion DoHoming(void)
{
    if (s homing mode == 2U) {
        // 方式 2: 速度模式 + 电流限制 (碰撞回零)
        X V2 Vel LC Control(MOTOR ADDR, s homing dir, 100,
                           (float)s homing vel rpm, false,
                           s homing current limit ma);
    } else {
        // 方式 1/3: 电机内置回零
        X V2 Origin Modify SL RP(MOTOR ADDR, false, 300);
        HAL Delay(150);
        X V2 Origin Trigger Return(MOTOR ADDR, s homing mode, false);
    }
}
```

9.4 移动函数

```
void Motion MoveTo(float pos deg, float vel rpm, uint16 t acc, bool is absolute)
{
    float abs pos = pos deg;
    uint8 t dir = (pos deg >= 0.0f) ? 0 : 1;
    uint8 t raf = is absolute ? 1 : 2; // 1= 绝对, 2= 相对
```

```

// X V2 要求位置为绝对值，方向单独指定
abs pos = (pos deg >= 0.0f) ? pos deg : -pos deg;

// 调用 X V2 梯形位置模式
X V2 Traj Pos Control(MOTOR ADDR, dir, acc, acc, vel rpm, abs pos, raf, false);

// 注册超时跟踪
SM TrackPendingCmd(0xFD);
SM TrackActionCmd(0xFD, timeout);
}

```

9.5 循环运行

循环配置结构

```

typedef struct {
    float pos deg;    // 目标位置
    float vel rpm;    // 运行速度
    uint16 t acc;     // 加速度
    uint16 t dwell ms; // 停留时间 (ms)
} CyclePoint t;

typedef struct {
    CyclePoint t points[25]; // 最多 25 个点
    uint8 t point count;    // 实际点位数
    uint16 t total cycles;  // 总循环次数 (0= 无限)
    uint16 t current cycle; // 当前循环计数
    uint8 t current point;  // 当前目标点索引
} CycleConfig t;

```

启动循环

```

void Motion StartCycle(CycleConfig t cfg)
{
    if (cfg == NULL || cfg->point count == 0) return;

    memcpy(&g cycle cfg, cfg, sizeof(CycleConfig t));
    g cycle cfg.current cycle = 1;
    g cycle cfg.current point = 0;

    // 移动到第一个点
    CyclePoint t pt = &g cycle cfg.points[0];
    Motion MoveTo(pt->pos deg, pt->vel rpm, pt->acc, true);
}

```

推进到下一点

```

void Motion CycleNextPoint(void)
{
    g cycle cfg.current point++;

    // 判断是否完成一圈
    if (g cycle cfg.current point >= g cycle cfg.point count) {
        g cycle cfg.current point = 0;

        // 判断是否完成全部循环
        if (g cycle cfg.total cycles > 0) {

```

```

        if (g cycle cfg.current cycle >= g cycle cfg.total cycles) {
            SM PostEvent(EV CYCLE DONE);
            return;
        }
        g cycle cfg.current cycle++;
    }
}

// 移动到下一点
CyclePoint t pt = &g cycle cfg.points[g cycle cfg.current point];
Motion MoveTo(pt->pos deg, pt->vel rpm, pt->acc, true);
}

```

9.6 X V2 驱动函数

函数	功能
X V2 Vel LC Control()	速度模式 + 电流限制
X V2 Traj Pos Control()	梯形位置模式
X V2 Stop Now()	立即停止
X V2 En Control()	使能控制
X V2 Origin Trigger Return()	触发回零

9.7 代码位置速查

内容	文件	行号
Motion DoHoming()	motion.c	45-56
Motion MoveTo()	motion.c	103-119
Motion Stop()	motion.c	121-125
Motion StartCycle()	motion.c	166-178
Motion CycleNextPoint()	motion.c	186-216

10. 第八阶段：安全监测

10.1 学习目标

理解异常是如何检测和处理的。

10.2 关键文件

App/Src/safety.c -> Safety Check() (第24-29行)

App/Src/safety.c -> Safety UpdateOafFlags() (第39-57行)

10.3 Safety Check() 函数

```

ErrorCode t Safety Check(void)
{
    s last read tick = g sys tick;

    // 读取电机 OAF 标志位
    X V2 Read Sys Params(MOTOR ADDR, S OAF);
}

```

```

    return s active error;
}

```

关键理解：Safety Check() 只发送读取命令，实际的状态更新在 CAN 接收回调中完成。

10.4 Safety UpdateOafFlags() 函数

```

ErrorCode t Safety UpdateOafFlags(uint8 t origin flags, uint8 t motor flags)
{
    s motor flags = motor flags;
    s origin flags = origin flags;

    if ((origin flags & HOME FLAG OVERCURRENT) != 0U) {
        s active error = ERR OVERCURRENT;
    }
    else if ((origin flags & HOME FLAG OVERTEMP) != 0U) {
        s active error = ERR OVERTEMP;
    }
    else if ((motor flags & MOTOR FLAG STALL PRO) != 0U) {
        s active error = ERR STALL PROTECT;
    }
    else if ((origin flags & HOME FLAG FAILED) != 0U && g sm state == SYSTEM HOMING) {
        s active error = ERR HOME FAILED;
    }
    else if (s active error != ERR ESTOP) {
        s active error = ERR NONE;
    }

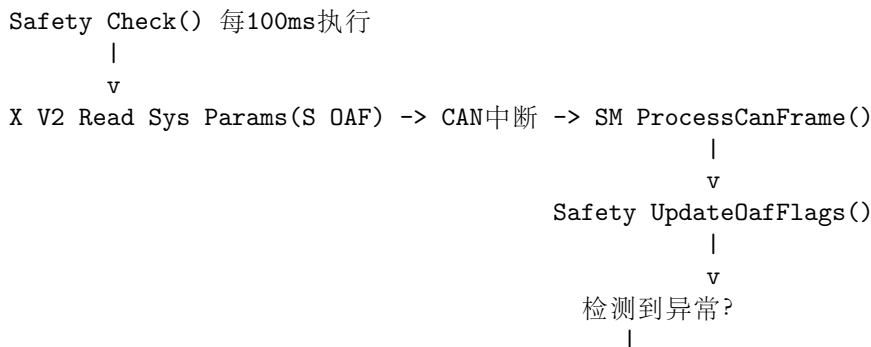
    return s active error;
}

```

10.5 错误类型

错误码	名称	说明
0x00	ERR NONE	无错误
0x01	ERR STALL PROTECT	堵转保护
0x02	ERR OVERCURRENT	过流保护
0x03	ERR OVERTEMP	过热保护
0x04	ERR HOME FAILED	回零失败
0x05	ERR POS ERROR	位置误差过大
0x06	ERR ESTOP	急停触发
0x07	ERR MOTION TIMEOUT	动作执行超时

10.6 错误处理流程





10.7 错误清除

```

void Safety ClearError(void)
{
    if (g_sm state != SYSTEM ESTOP) {
        X V2 Reset Clog Pro(MOTOR ADDR); // 解除堵转保护
    }
    s_active_error = ERR_NONE;
}

```

关键理解：错误必须双重处理：1. 主控本地立即响应（停止电机）2. 同步告知上位机（发送错误事件）

10.8 代码位置速查

内容	文件	行号
Safety Init()	safety.c	16-22
Safety Check()	safety.c	24-29
Safety UpdateOafFlags()	safety.c	39-57
Safety ClearError()	safety.c	31-37

11. 第九阶段：中断系统

11.1 学习目标

理解中断和主循环的分工协作。

11.2 关键文件

Core/Src/stm32g4xx_it.c -> 各中断回调函数

11.3 四种中断

FDCAN 中断（CAN 接收）

```

void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef hfdcan, uint32_t RxFifo0ITs)
{
    // 接收 CAN 帧 -> 存入环形缓冲区
    CanRxBuf_Push(rxHeader.Identifier, dlc_bytes, rxData);
}

```

职责：接收 CAN 帧，存入环形缓冲区 不做：解析帧内容、调用状态机、发送命令

USART 中断（串口接收）

```
void USART1_IRQHandler(void)
{
    // RXNE 中断: 接收字节 -> 存入环形缓冲区
    if (UART_FLAG_RXNE) {
        g_usart_rx_buf[g_usart_rx_head] = byte;
        g_usart_rx_head = (g_usart_rx_head + 1) % USART_RX_BUF_SIZE;
    }
}
```

职责：接收字节，存入环形缓冲区 不做：解帧、路由命令

TIM6 中断（系统 tick）

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef htim)
{
    if (htim->Instance == TIM6) {
        SM_TickIncrement(); // 递增系统 tick
    }
}
```

职责：递增系统 tick 计数器（1kHz） 不做：直接读电机、调用 CAN 发送

EXTI 中断（急停按钮）

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == ESTOP_Pin) {
        g_estop_triggered = (读取引脚电平 == LOW);
    }
}
```

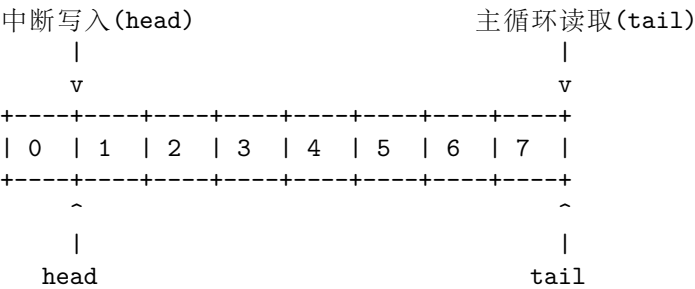
职责：设置急停标志 不做：调用状态机、直接操作电机

11.4 中断与主循环分工

机制	承担任务	不在其中做的事
FDCAN 中断	接收 CAN 帧存入缓冲区	不解析、不调用状态机
USART 中断	接收字节存入缓冲区	不解帧、不路由命令
TIM6 中断	递增系统 tick	不读电机、不发 CAN
EXTI 中断	设置急停标志	不调用状态机、不动电机
主循环	消费缓冲区、执行状态机、发送命令	不阻塞等待

设计原则：中断只做收和记，主循环做想和发。

11.5 环形缓冲区设计



- 写入: `buf[head] = data; head = (head + 1) % SIZE;`
- 读取: `data = buf[tail]; tail = (tail + 1) % SIZE;`
- 满检测: `(head + 1) % SIZE == tail`
- 空检测: `head == tail`

11.6 代码位置速查

内容	文件	行号
CAN 中断回调	stm32g4xx_it.c	258-275
USART 中断	stm32g4xx_it.c	294-315
TIM6 中断回调	stm32g4xx_it.c	364-369
EXTI 中断回调	stm32g4xx_it.c	334-345

12. 第十阶段：完整数据流

12.1 学习目标

将所有模块串联，理解完整的数据流。

12.2 上位机到电机（指令流）

```

USART1_RX中断
    |
    v
环形缓冲区
    |
    v
Protocol_Parse() <-- 逐字节状态机
    |
    v
ProtocolFrameHandler() <-- 命令分发
    |
    v
SM_PostEvent() <-- 产生系统事件
    |
    v
SM_EnterState() <-- 状态转换
    |
    v
Motion_xxx() <-- 工艺动作封装
    |
    v
X_V2_xxx() <-- 电机驱动库
    |
    v
can_SendCmd() <-- FDCAN发送
    |
    v
电机

```

12.3 电机到上位机（应答流）

FDCAN1_RX中断

```

|
v
CanRxBuf Push() <-- 存入环形缓冲区
|
v
CanRxBuf Pop() <-- 主循环取出
|
v
SM ProcessCanFrame() <-- 解析应答码/数据帧
|
+--> 0x02(ACK) --> Protocol SendEvent(ACK RECV) --> 上位机
|
+--> 0x9F(DONE) --> SM PostEvent(MOVE DONE) --> 状态机
|
|                                     |
|                                     +--> Protocol SendEvent(ACT DONE) --> 上位机
|
+--> 0xE2(ERROR) --> SM PostEvent(ERROR DETECTED) --> 状态机
|
|                                     |
|                                     +--> Protocol SendEvent(MOTOR ERROR) --> 上位机
|
+--> 数据帧 --> SM ProcessMotorData() --> Protocol SendEvent(MOTOR DATA)

```

12.4 三条核心链路

链路 A: 上电自动初始化与回零

```

上电 -> INIT -> 配置电机 -> HOMING -> 等待完成 -> IDLE
|
+--> HOME START --> 上位机
+--> HOME DONE/FAILED --> 上位机

```

链路 B: 指令送达确认

```

上位机命令 -> 校验 -> 转发电机 -> 电机回0x02 -> ACK RECV --> 上位机
|
+--> 电机回0xE2 --> MOTOR ERROR --> 上位机

```

链路 C: 动作完成确认

```

电机执行中 -> 电机回0x9F --> MOVE DONE --> IDLE
|
+--> ACT DONE --> 上位机

电机执行中 -> 电机回0xE2 --> ERROR --> 停止电机
|
+--> MOTOR ERROR --> 上位机

```

12.5 典型操作流程

移动命令流程

1. 上位机发送 MOVE ABS 命令
2. Protocol Parse() 解析帧
3. ProtocolFrameHandler() 处理命令
 - 状态不是IDLE? -> 返回 BUSY
 - 参数错误? -> 返回 FAIL
4. SM PostEvent(EV CMD MOVE)
5. SM EnterState(SYSTEM RUNNING)

- Motion MoveTo() 发送移动命令
 - SM TrackPendingCmd() 跟踪ACK
 - SM TrackActionCmd() 跟踪完成
6. 等待电机应答
- 收到 0x02 -> ACK RECV -> 上位机
7. 等待动作完成
- 收到 0x9F -> MOVE DONE -> IDLE
 - 收到 0xE2 -> ERROR

错误处理流程

1. Safety Check() 检测到异常
2. Safety UpdateOafFlags() 返回错误码
3. SM PostEvent(EV ERROR DETECTED)
4. SM EnterState(SYSTEM ERROR)
 - Motion Stop() 停止电机
5. Protocol SendEvent(MOTOR ERROR) 通知上位机
6. 等待上位机发 RESET ERROR
7. SM PostEvent(EV CMD RESET ERROR)
8. SM EnterState(SYSTEM IDLE)
 - Safety ClearError() 清除错误
 - Protocol SendResponse(OK) 回复上位机

13. 附录：常见问题

13.1 常见问题

Q1: 系统上电后不回零

可能原因: - 电机未使能 - CAN 通信异常 - 回零参数配置错误

调试方法: 1. 检查电机是否使能 2. 检查 CAN 通信是否正常 (查看 g can rx count) 3. 检查回零参数

Q2: 移动命令无响应

可能原因: - 状态不是 IDLE (返回 BUSY) - 参数错误 (返回 FAIL) - CAN 通信超时

调试方法: 1. 发送 READ STATUS 命令查看当前状态 2. 检查参数是否在有效范围内 3. 检查 CAN 通信是否正常

Q3: 系统进入 ERROR 状态

可能原因: - 电机异常 (堵转、过流、过热) - 回零失败 - 动作超时

调试方法: 1. 发送 READ STATUS 命令查看错误码 2. 检查电机状态标志位 3. 发送 RESET ERROR 命令清除错误

13.2 调试技巧

使用 **READ STATUS** 命令

发送: AA 01 10 10 55

响应: AA 04 10 00 [STATE] [ERROR] [CRC8] 55

- STATE: 0=INIT, 1=IDLE, 2=HOMING, 3=RUNNING, 4=CYCLING, 5=ERROR, 6=ESTOP
- ERROR: 0= 无, 1= 堵转, 2= 过流, 3= 过热, 4= 回零失败, 5= 位置误差, 6= 急停

13.3 命令速查表

命令	完整帧	说明
HOME	AA 01 01 01 55	触发回零
STOP	AA 01 04 04 55	立即停止
RESET ERROR	AA 01 07 07 55	清除错误
CYCLE STOP	AA 01 06 06 55	停止循环
READ STATUS	AA 01 10 10 55	读取状态

结语

通过以上 10 个阶段的学习，你应该已经对 M Project 有了全面的理解。这个项目的核心是：

1. 状态机驱动：所有逻辑都在状态机中管理
2. 事件驱动：通过事件触发状态转换
3. 双向中介：主控作为上位机和电机之间的桥梁
4. 安全第一：错误必须双重处理（本地响应 + 上位机通知）

祝你在比赛中取得好成绩！